Engineering Assignment — Creator Support Bot

Background

You're building a platform that connects brands with content creators for paid social media campaigns. Brands post jobs, creators apply, and the platform handles content tracking, payments, and payouts.

Currently, creators message our support team (via Slack DMs) with repetitive questions — things like "how much do I get paid?", "what should I post?", "how do I connect my bank account?". Our account manager answers ~50-60 of these per day, and most have straightforward answers that could be looked up from existing data.

Your task: Design an AI-powered support bot that auto-responds to creator questions on the platform, using the creator's actual data to give personalized answers — and knows when to escalate to a human.

Current State

We have a basic messaging system on the platform:

```
Creator sends message → stored in database → Slack notification to support team → human replies man
```

Messages are organized into conversations. Each creator has a "general support" thread where they can message the team. There is no auto-response — every message requires a human.

Example Creator Questions

Here are real questions our support team receives daily:

Question	What's needed to answer it
"How much do I get paid?"	Creator's pay rate, pay structure (per-video vs monthly), contract

Question	What's needed to answer it
"When do I start posting?"	Creator's warmup status (not started / in progress / complete)
"What do I post?"	Campaign brief, content format, reference videos
"How many videos do I need to post?"	Campaign video quota, posting frequency
"How do I set up Spark Codes?"	Static instructions (same for everyone)
"My account won't sync"	Platform-specific troubleshooting, may need escalation
"How long is warmup?"	Static instructions + creator's current warmup day
"How do I connect my bank account?"	Static instructions for Stripe onboarding
"When is my next payment?"	Creator's payment history, pending balance
"Can I post on Instagram too?"	Campaign's platform list (TikTok, Instagram, YouTube)
"I haven't been paid for my last video"	**Always escalate** — payment disputes need a human
"Am I going to be dropped?"	**Always escalate** — judgment calls need a human

Notice that some answers require looking up the creator's specific data, some require looking up their campaign's data, some are universal (same for everyone), and some should always be escalated to a human.

Technical Constraints

Constraint	Value
Stack	Next.js (App Router) + PostgreSQL (Supabase)
AI model	Claude API (Anthropic) — you can use any model size
Response time target	< 5 seconds for a bot reply

Constraint	Value
Cost target	< $0.01 per response
Message volume	~50-100 creator messages per day (growing)
Creators	~500 active across all campaigns
Campaigns	~10 active simultaneously

Data Model (simplified)

```
creators
├── id
├── name
├── email
├── campaign_id (FK)
├── pay_rate (nullable — dollars per video)
├── pay_structure (per_video | monthly | null)
├── contract_signed (boolean)
├── bank_connected (boolean)
├── warmup_status (not_started | in_progress | complete)
├── warmup_day (integer)
├── total_videos_posted
├── total_paid (cents)
└── last_posted_at

campaigns
├── id
├── brand_name
├── brief_url
├── platforms (tiktok | instagram | youtube)[]
├── posting_frequency (e.g., "once per day per platform")
├── video_quota (integer)
├── content_format (text description)
├── hashtags (text[])
└── active (boolean)

conversations
├── id
├── creator_id (FK)
├── type (support | campaign)
└── last_message_at

messages
```

```
├── id
├── conversation_id (FK)
├── sender_id
├── message_text
├── is_system_message (boolean)
├── created_at
└── read_by (JSON)
```

Requirements

Core functionality

- When a creator sends a message in their support thread, the bot should attempt to answer it automatically
- Answers must be personalized — use the creator's actual data, not generic responses
- The bot should know when it CAN'T or SHOULDN'T answer, and escalate to a human
- Escalations should notify the support team (e.g., via Slack) with context about what was asked

Quality

- Responses should be friendly and concise — match the tone of a helpful teammate, not a corporate FAQ
- The bot should never fabricate data — if a field is null or missing, acknowledge it and escalate
- The bot should never make judgment calls (content quality, drop decisions, payment disputes)

Observability

- Support team needs to see what the bot said and whether it was helpful
- There should be a way to improve the bot over time based on what it gets wrong

Your Task

Design a system that meets these requirements.

Deliverables

1. Architecture Overview (1-2 pages)

- How does the bot decide what to answer vs. escalate?

- How does it fetch and inject the right context for each question?
- Where does the bot logic run in the request lifecycle?
- How do you handle multi-campaign creators?

2. Prompt Engineering

- Show your system prompt design
- How do you structure context injection? (What data goes into the prompt, in what format?)
- How do you prevent hallucination and enforce escalation?
- Include at least 3 example prompt → response pairs

3. Data & API Design

- What queries need to run before the bot can respond?
- Any schema changes needed?
- API route design — how does the bot hook into the existing message flow?

4. Escalation Logic

- Define clear rules for when the bot should NOT answer
- How does the escalation notification work?
- How does the human pick up where the bot left off?

5. Trade-offs & Edge Cases

- What does your design optimize for? What does it sacrifice?
- How do you handle: creator on multiple campaigns? Bot gives wrong answer? Creator tries to manipulate the bot?
- How would you measure whether the bot is actually helping?

Bonus (optional)

- How would you build a feedback loop to improve the bot over time?
- How would you handle multi-language support (creators span 9+ countries)?

How You Work

We want to see how you use AI to approach this problem. Using AI tools (Claude, ChatGPT, Copilot, Gemini — whatever you prefer) is encouraged, not penalized. Include a short section describing:

- How you used AI during this assignment

- Where it helped and where you had to override or correct it
- 2-3 screenshots of interesting AI interactions

This is not about catching you — we use AI heavily in our engineering workflow and want to see that you can leverage it effectively.

Evaluation Criteria

Criteria	Weight
Architecture — does the design actually solve the problem?	25%
Prompt engineering — smart context injection, hallucination prevention	25%
Escalation logic — knows what it doesn't know	20%
Clarity of thinking and communication	15%
Practical and buildable (not over-engineered)	15%

Timeline

48 hours from receiving this assignment.

We're evaluating your thinking process and judgment, not your ability to write production code under time pressure. Pseudocode, diagrams, and clear explanations are more valuable than polished code that doesn't demonstrate understanding.

Submission

Send your completed assignment as a PDF or Google Doc to theo@8x.social within 48 hours. Once submitted, text me on WhatsApp or LinkedIn to confirm, and schedule a call with me for the following day so we can discuss your submission together.